

Apply filters to SQL queries

Project description

As part of my responsibility to enhance the system's security, I diligently worked on investigating potential security issues and updating employee computers when necessary. To accomplish these tasks, I employed SQL queries with filters, which allowed me to address security-related concerns effectively. The following steps demonstrate how I utilized SQL to perform these tasks.

Retrieve after hours failed login attempts

There was a potential security incident that occurred after business hours (after 18:00)

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE login_time > '18:00' AND success = FALSE;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0

The provided query filters for failed login attempts that occurred after 18:00. By selecting data from the "log_in_attempts" table and using conditions within a WHERE clause, the query focuses on retrieving only the unsuccessful login attempts that took place after 18:00.

Retrieve login attempts on specific dates

A suspicious event occurred on 2022-05-09. Any login activity that happened on 2022-05-09 or on the day before needs to be investigated.

The following code demonstrates how I created a SQL query to filter for login attempts that occurred on specific dates.

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	0
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0

The provided query retrieves login attempts that took place on either 2022-05-09 or 2022-05-08. By selecting data from the "log_in_attempts" table and utilizing a WHERE clause with the OR operator,

The query narrows down the results to include only login attempts on those specific dates.

Retrieve login attempts outside of Mexico

After investigating the organization's data on login attempts, I believe there is an issue with the login attempts that occurred outside of Mexico.

```
MariaDB [organization]> SELECT *  
-> FROM log_in_attempts  
-> WHERE NOT country LIKE 'MEX%';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	0
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	0

The provided query retrieves login attempts that occurred in countries other than Mexico. By selecting data from the "log_in_attempts" table and using a WHERE clause with the NOT operator, the query filters out login attempts associated with Mexico. To accomplish this, the LIKE operator is employed with the pattern "MEX%," as the dataset represents Mexico as "MEX" and "MEXICO." The "%" sign signifies any number of unspecified characters when used in conjunction with LIKE.

Retrieve employees in Marketing

My team wants to update the computers for certain employees in the Marketing department. To do this, I have to get information on which employee machines to update.

```
MariaDB [organization]> SELECT *  
-> FROM employees  
-> WHERE department = 'Marketing' AND office LIKE 'East%';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1052	a192b174c940	jdarosa	Marketing	East-195
1075	x573y883z772	fbautist	Marketing	East-267

The query retrieves employees who belong to the Marketing department and work in the East building. By selecting data from the employees table and using a WHERE clause with AND, the query filters for employees in the Marketing department and in the office location starting with "East." The condition department = 'Marketing' filters for employees in the Marketing department, while the condition office LIKE 'East%' filters for employees in the East building.

Retrieve employees in Finance or Sales

The machines for employees in the Finance and Sales departments also need to be updated. Since a different security update is needed, I have to get information on employees only from these two departments.

```
MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Finance' OR department = 'Sales';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170

The query retrieves employees who belong to either the Finance or Sales departments. By selecting data from the employees table and utilizing a WHERE clause with the OR operator, the query filters for employees in either the Finance or Sales department. The conditions department = 'Finance' and department = 'Sales' respectively filter for employees from the Finance and Sales departments.

Retrieve all employees not in IT

My team needs to make one more security update on employees who are not in the Information Technology department. To make the update, I first have to get information on these employees.

```
MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE NOT department = 'Information Technology';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434

The query retrieves employees who do not belong to the Information Technology department. By selecting data from the employees table and utilizing a WHERE clause with the NOT operator, the query filters out employees who are in the Information Technology department. This allows for the retrieval of employees from other departments.

Summary

In my SQL queries, I effectively employed filters to extract specific information from the log_in_attempts and employees tables. By utilizing operators such as AND, OR, and NOT, I could refine the search criteria and retrieve the desired data. Additionally, I leveraged the LIKE operator along with the percentage sign (%) wildcard to filter for patterns within the data.

Through these techniques, I was able to narrow down the results and obtain targeted information related to login attempts and employee machines. The combination of table selection, operator usage, and wildcard filtering allowed me to efficiently gather the data.